

Lastenheft

SASD Desktop Secret Manager

Überarbeitete Fassung aus Dokumenten, Projektchats und Repository-Stand

Windows-Desktop-Anwendung für lokale, verschlüsselte Mehrfach-Tresore, strukturierte technische Secrets, HTTPS/TLS-Zertifikatsverwaltung und realistische Password-Safe-Interop

Metadatum	Wert
Dokumenttyp	Lastenheft, überarbeitete Fassung
Projekt	SASD Desktop Secret Manager / lokaler Passwort- und Secret-Manager
Dokumentziel	Konsolidierung der bisherigen Dokumente, Projektchats, Roadmap und Repository-Hinweise zu einer bereinigten fachlichen Anforderungsbasis.
Adressaten	Auftraggeber, Produktentscheidung, Architektur, Entwicklung, Test, spätere Release-Planung
Status	Arbeits- und Entscheidungsgrundlage; kein produktives Sicherheitsfreigabedokument
Stand	18.05.2026
Quellenbasis	Hochgeladene Lastenheft-/Pflichtenheft-/Architekturdokumente, Feature-Collation, Roadmap, Projektchats und aktueller GitHub-README-Stand.

Leitgedanke

Dieses Lastenheft fasst die bisherigen Fassungen zusammen, bereinigt Überschneidungen und ergänzt die im Chat hinzugekommenen Punkte ausdrücklich. Besonders die Verwaltung von HTTPS/TLS-Zertifikaten wird nicht mehr nur als beiläufiger Zertifikatsbezug verstanden, sondern als eigener fachlicher Themenbereich mit klarer Versionszuordnung.

Inhaltsverzeichnis

1. Management Summary
2. Dokumentzweck und Quellenbasis
3. Ausgangssituation und Problemstellung
4. Zielbild des Produkts
5. Einsatzkontext, Stakeholder und Schutzannahmen
6. Fachlicher Leistungsumfang
7. Fachliches Datenmodell
8. Tresorverwaltung und Dateikonzept
9. Organisation, Navigation, Suche und Bedienfluss
10. Sicherheitsleitlinien und Schutzgrenzen
11. Recovery ohne Backdoor
12. Interoperabilität, Migration, Import und Export
13. Release- und Versionslogik
14. Funktionaler Anforderungskatalog
15. Nichtfunktionale Anforderungen
16. Bewusste Nicht-Ziele und vertagte Themen
17. Aktueller Prototypstand
18. Abnahmekriterien für V1
19. Offene Punkte für das anschließende Pflichtenheft
20. Quellen- und Entscheidungsregister
21. Schlussbemerkung

Hinweis

Die Seitenzahlen werden im PDF korrekt angezeigt; die Inhaltsstruktur ist als statisches, bewusst schlichtes Verzeichnis angelegt.

1. Management Summary

Aus den bisherigen Dokumenten und Projektchats ergibt sich ein klares Zielbild: Der SASD Desktop Secret Manager soll kein schlichter 1:1-Nachbau von Password Safe und keine bloße Passwortliste werden. Gesucht ist ein eigenständiger lokaler Secret Manager, der klassische Logins, technische Betriebszugänge und sicherheitsrelevante Kontextdaten sauber in mehreren voneinander unabhängigen verschlüsselten Tresoren verwaltet.

Der fachliche Schwerpunkt liegt auf einem offline-first nutzbaren Windows-Desktop-Produkt. Die Anwendung muss mehrere Tresore, Gruppen, Untergruppen, Tags, Suche, strukturierte Zusatzfelder, sichere Anzeige- und Kopiermechanismen, belastbare Dateiverwaltung und eine realistische Interoperabilität zu Password Safe unterstützen. Das interne Datenmodell bleibt dabei führend; Fremdformate werden angebunden, dürfen aber das Produkt nicht fachlich einschränken.

Neu gegenüber den älteren Fassungen wird die Verwaltung von HTTPS/TLS-Zertifikaten ausdrücklich als eigener Anwendungsfall aufgenommen. Zertifikate sollen nicht nur als Notiz oder beiläufiger Zertifikatsbezug auftauchen, sondern als strukturierter Secret-Kontext mit Domain, Common Name, Subject Alternative Names, Aussteller, Laufzeit, Fingerprint, Ablageort, Deployment-Ziel, privatem Schlüsselbezug und Erneuerungshinweisen modellierbar sein.

Kernentscheidung

Das Produkt bleibt lokal, sicherheitsbewusst und für den Einzelplatz beziehungsweise die kleine SASD-Arbeitsumgebung optimiert. Team-Sharing, Cloud-Sync, Mobile-Clients und Browser-Autofill sind keine V1-Ziele. Sie dürfen später diskutiert werden, aber nicht den lokalen Kern verwässern.

2. Dokumentzweck und Quellenbasis

Dieses Lastenheft beschreibt die fachlichen Anforderungen an das Zielprodukt. Es ist bewusst lösungsnah genug, um Missverständnisse bei Architektur und Umsetzung zu vermeiden, bleibt aber ein Lastenheft: Es beschreibt, was das Produkt leisten soll, warum es gebraucht wird und welche Grenzen gelten. Detaillierte technische Umsetzung, Klassenstruktur und konkrete Algorithmen gehören weiterhin in Pflichtenheft und Architekturdokument.

Die Quellenbasis besteht aus den hochgeladenen Lastenheft-, Pflichtenheft-, Architektur-, Featuremap-, Roadmap- und Vergleichsdokumenten, dem aktuellen GitHub-Repository README sowie den relevanten Projektchats. Der bisherige Prototyp wird berücksichtigt, aber nicht mit dem Zielzustand verwechselt. Dieses Dokument ist damit eine Konsolidierung und redaktionelle Bereinigung, nicht nur eine Kopie einer früheren Fassung.

- Frühere Lastenheft- und Architekturfassungen mit Fokus auf lokalem Secret Manager, Mehrtresor-Konzept und Password-Safe-Interop.
- Fortgeschriebene Lastenheft- und Pflichtenheftfassung mit Versionsregister, bewussten Nicht-Zielen und Prototypstand.
- Architekturdokument V1 mit Ist-/Soll-Abgleich des Repository-Stands.
- Feature-Collation der betrachteten Passwortmanager als Inspirations- und Abgrenzungsquelle.
- Roadmap und Projektchats mit später hinzugekommenen Punkten, insbesondere Recovery ohne Backdoor und HTTPS/TLS-Zertifikatsverwaltung.

3. Ausgangssituation und Problemstellung

Im SASD-Kontext werden nicht nur gewöhnliche Web-Logins verwaltet. Benötigt wird eine Anwendung für Hosting-Zugänge, Datenbankverbindungen, Webpace- und FTP/SFTP-Zugänge, Mail-Konten, GitHub-Zugänge, Server- und Infrastrukturzugriffe, API-Schlüssel, Lizenzschlüssel, technische Tokens, sichere Notizen sowie künftig strukturierte HTTPS/TLS-Zertifikatsinformationen. Ein Datensatz aus Titel, Benutzername und Passwort reicht dafür fachlich nicht aus.

Die bisherigen Daten können organisatorisch sehr unterschiedlich sein: private Zugänge, SASD-interne Zugänge, providerbezogene Informationen wie IONOS, kundenbezogene Zugänge, Entwicklungs- und Produktionszugänge oder archivierte Bestände. Diese Inhalte dürfen nicht in einer einzigen ungeordneten Liste verschwinden. Sie benötigen getrennte Tresore, klare Gruppen, zusätzliche Tags und eine leistungsfähige Suche.

Gleichzeitig existiert eine fachliche Nähe zu Bruce Schneiers Password Safe beziehungsweise PWSafe. Bestehende Daten und Nutzungsgewohnheiten sollen nicht ignoriert werden. Dennoch wäre ein starrer 1:1-Klon fachlich zu eng. Das Zielprodukt soll ehrliche Interoperabilität bieten, aber sein eigenes Datenmodell nicht auf die Grenzen eines Fremdformats reduzieren.

4. Zielbild des Produkts

Ziel ist eine lokal nutzbare Windows-Desktop-Anwendung, die verschlüsselte Tresore für Passwörter, Zugangsdaten, Tokens, Zertifikatsinformationen und strukturierte technische Secrets verwaltet. Jeder Tresor ist eine eigenständige Schutzdomäne mit eigenem Master-Passwort, eigener Datei, eigenen Metadaten und eigener organisatorischer Struktur.

Ein Eintrag beschreibt einen primären Zugang oder ein primäres Secret fachlich sauber. Er kann über Typ, Titel, Benutzerkennung, Secret-Wert, Notizen, Gruppe, Tags und strukturierte Zusatzfelder so beschrieben werden, dass er im Alltag auffindbar und verständlich bleibt. Verwandte Zugänge werden über Gruppen, Tags und spätere Referenzen verbunden, nicht durch Vermischen mehrerer fachlicher Objekte in einem Sammeltext.

Die Bedienung soll ruhig, professionell und technisch nachvollziehbar sein: links Tresor- und Gruppenstruktur, zentral Suche und Eintragsliste, rechts beziehungsweise in einem klaren Bereich Detailanzeige und sichere Copy-/Reveal-Aktionen. Kontextmenüs, Tastaturbedienung und Drag-and-Drop ergänzen den Ablauf, ohne die Oberfläche mit zu vielen Direktbuttons zu überladen.

5. Einsatzkontext, Stakeholder und Schutzannahmen

Aspekt	Fachliche Einordnung
Primärer Einsatz	Lokaler Windows-Desktop, Einzelplatznutzung, offline-first, keine Pflicht zu Cloud-Konto oder Webdienst.
Nutzerkreis	Technisch versierter Einzelanwender, SASD-GmbH-nahe Arbeit, später ggf. kleine interne Nutzung oder vorbereitete Firmenprozesse.
Datenarten	Private, SASD-interne, providerbezogene, kundenbezogene und archivierte Geheimnisse und Zugangsinformationen.

Aspekt	Fachliche Einordnung
Schutzannahme	Das Betriebssystem gilt grundsätzlich als vertrauenswürdig. Gegen vollständig kompromittierte Hosts, Keylogger oder aggressive Malware kann die Anwendung allein keinen vollständigen Schutz versprechen.
Produktcharakter	Sicherheitsrelevantes Arbeitsprodukt mit Prototyp-/Entwicklungsstand; keine Freigabe für alleinige produktive Verwaltung echter kritischer Secrets vor V1-Härtung.

6. Fachlicher Leistungsumfang

6.1 Gegenstand der Verwaltung

Die Anwendung muss klassische und technische Secret-Kontexte verwalten. Dazu gehören mindestens Logins, Datenbankverbindungen, Mail-Konten, Hosting-Zugänge, FTP/SFTP-Zugänge, Server- und Infrastrukturzugänge, API-Schlüssel, Lizenzinformationen, sichere Notizen, frei definierte Custom-Einträge und HTTPS/TLS-Zertifikatskontexte.

Secret-Kontext	Typische fachliche Informationen
Login	URL, Benutzername, Passwort, Notizen, 2FA-Hinweise, betroffener Dienst.
Datenbank	Host, Port, Datenbankname, Schema, Benutzer, Passwort, SSL/TLS-Hinweis, Umgebung.
Mail	E-Mail-Adresse, IMAP/SMTP-Hosts, Ports, TLS-Einstellung, Benutzername, Passwort.
Hosting/Webspace	Provider, Paket, Login, Zielsystem, Vertrags-/Kundenbezug, technische Notizen.
FTP/SFTP	Host, Port, Protokoll, Benutzer, Passwort oder Schlüsselhinweis, Zielpfad.
Server/Infrastruktur	Hostname/IP, Protokoll, Port, Rolle, Umgebung, Benutzer, Passwort oder Schlüsselbezug.
API/Token	Endpoint, Client-ID, Secret, Token, Scope, Ablaufdatum, verantwortliches System.
Lizenz	Produkt, Lizenzschlüssel, Hersteller, Ablaufdatum, Vertragsbezug, Nutzer-/Gerätezuordnung.
HTTPS/TLS-Zertifikat	Domain/CN, SANs, Aussteller, Seriennummer, Fingerprint, Laufzeit, privater Schlüsselbezug, PFX/PEM/CRT-Hinweis, Deployment-Ort, Erneuerung.
Secure Note / Custom	Freie sicherheitsrelevante Informationen, die nicht sinnvoll in eines der Standardmuster passen.

6.2 HTTPS/TLS-Zertifikatsverwaltung

Die Zertifikatsverwaltung wird als eigener fachlicher Themenbereich aufgenommen. Ziel ist nicht sofort eine vollständige PKI- oder ACME-Verwaltung, sondern zunächst die saubere Erfassung und Wiederauffindbarkeit von Zertifikatsinformationen im Kontext von Servern, Domains, Webpace und Anwendungen. Ein Zertifikatseintrag darf deshalb nicht nur aus einer Notiz bestehen, sondern benötigt strukturierte Felder.

Feldgruppe	Muss/Soll-Inhalte
Identität	Anzeigenname, Zertifikatstyp, Domain, Common Name, Subject Alternative Names, Wildcard-Kennzeichnung.
Aussteller und Laufzeit	Issuer/CA, Seriennummer, Fingerprint/Thumbprint, gültig von, gültig bis, Erneuerungsfrist.
Technische Ablage	Format-Hinweis wie PEM/CRT/CER/PFX/P12, Dateipfad oder Speicherort, Deployment-Ziel, Server/Hosting-Bezug.
Geheime Bestandteile	Privater Schlüssel, PFX/P12-Passwort, Key-Passphrase oder Zugriffsdaten nur als geheime Felder oder später als verschlüsselte Anhänge.
Beziehungen	Verknüpfung zu Server-, Hosting-, Domain-, Mail- oder API-Einträgen, damit ersichtlich bleibt, wo das Zertifikat verwendet wird.
Betrieb	Verantwortlicher, Erneuerungsverfahren, letzter Austausch, nächster Prüftermin, Hinweise zu Test-/Produktionsumgebungen.

Abgrenzung Zertifikate

V1 soll Zertifikate strukturiert beschreiben und geheime Bestandteile sicher behandeln können. Automatische Zertifikatserneuerung, ACME-Client-Funktionen, vollständige PKI-Verwaltung, Certificate Transparency Monitoring und automatische Server-Deployments sind spätere Themen und keine V1-Kernanforderung.

7. Fachliches Datenmodell

Das interne Datenmodell ist führend. Es muss genug Struktur bieten, damit technische Secrets nicht in Freitext verschwinden, darf den Nutzer aber nicht in starre Spezialdialoge zwingen. Der Kern besteht aus Vault, Entry, Group, Tag, CustomField und später optionalen Relationship-/Attachment-Konzepten.

Fachobjekt	Beschreibung	V1-Relevanz
Vault	Abgeschlossene Verwaltungseinheit mit Datei, Master-Passwort, Gruppen, Einträgen, Tags, Metadaten und Format-/Sicherheitsparametern.	Muss
Entry	Primärer Zugang oder primäres Secret mit Typ, Titel, Secret, Benutzerkennung, Notizen, Gruppe, Tags, Zeitstempeln und Zusatzfeldern.	Muss
Group	Hierarchische Ablage und Navigationsstruktur mit Root-/Tresorlogik und Untergruppen.	Muss
Tag	Freie Querzuordnung für Provider, Projekt, Umgebung, Kunde, Technologie oder Status.	Muss
CustomField	Strukturiertes Zusatzfeld mit Name, Wert, Feldart, Geheimkennzeichnung und Sortierreihenfolge.	Muss
Relationship	Spätere Relation zwischen Einträgen, z. B. Zertifikat gehört zu Server und Domain.	Soll später

Fachobjekt	Beschreibung	V1-Relevanz
Attachment	Spätere verschlüsselte Dateiablage für Zertifikatsdateien, Schlüsseldateien oder Nachweise.	Soll später

Zusatzfelder müssen mindestens Text, Secret, URL, Hostname, Port, E-Mail, Date, Number, Boolean und freie Typen unterstützen. Geheime Zusatzfelder werden wie normale Secrets behandelt: standardmäßig maskiert, nur bewusst kopierbar und niemals im Klartext in Logs oder Fehlermeldungen auszugeben.

8. Tresorverwaltung und Dateikonzept

Mehrere unabhängige Tresore sind ein Kernmerkmal des Produkts. Private, SASD-interne, providerbezogene, kundenbezogene und archivierte Bestände sollen getrennt gespeichert und getrennt entsperrt werden können. Pro Programmfenster ist für frühe Versionen genau ein aktiver, schreibbarer Tresor vorzusehen; parallele Schreibzugriffe auf dieselbe Datei sind konservativ zu behandeln.

Anforderung	Fachliche Beschreibung
Tresor anlegen	Neuer Tresor mit Anzeigename, Master-Passwort und leerer beziehungsweise minimaler Startstruktur.
Tresor öffnen	Bestehende Datei auswählen, Master-Passwort abfragen, bei Fehlern klare Meldung ohne Secret-Leak.
Speichern/Speichern unter	Nachvollziehbare, robuste Speicherlogik mit Schutz vor halbfertigen Dateien.
Schließen/Wechseln	Vor Verlust ungespeicherter Änderungen schützen und Inhalte aus der Oberfläche entfernen.
Backups	Backup-Dateien bleiben verschlüsselt. Unverschlüsselte Sicherungen sind unzulässig.
MRU-Liste	Zuletzt verwendete Tresore dürfen lokal gespeichert werden, aber ohne Master-Passwörter und ohne geheime Inhalte.

9. Organisation, Navigation, Suche und Bedienfluss

Ordnung entsteht im Produkt durch eine Kombination aus Gruppen, Tags, Typen, Zusatzfeldern und Suche. Ein reines Gruppenmodell wäre zu starr, ein reines Tag-Modell zu chaotisch. Die Anwendung muss beide Ordnungsformen bewusst kombinieren und sichtbar unterscheidbar machen.

- Der Tresor besitzt einen sichtbaren Root-/Tresorknoten, damit Hauptgruppen direkt auf oberster Ebene angelegt werden können.
- Einträge gehören genau einer Hauptgruppe an und können zusätzlich beliebig viele Tags besitzen.
- Suche und Filter berücksichtigen Titel, Benutzerkennung, Typ, Gruppe, Tags, Notizen und nicht geheime Zusatzfelder.
- Die Standardsuche ist case-insensitive; eine optionale Fallunterscheidung ist ein späteres Komfortthema.
- Einträge und Gruppen können per Kontextmenü und Drag-and-Drop organisiert werden, wobei riskante Aktionen bestätigt werden müssen.
- Tags sollen in Detailansichten anklickbar sein und in Filter- oder Suchaktionen überführt werden können.

10. Sicherheitsleitlinien und Schutzgrenzen

Die Anwendung ist sicherheitsrelevant. Sie muss insbesondere gegen Verlust oder Diebstahl der Tresordatei, versehentliche Offenlegung am Arbeitsplatz, dauerhaft gefüllte Zwischenablage, Datenverlust bei Speichervorgängen, unsichere Exporte und Secret-Leaks in Logs schützen. Gleichzeitig muss sie ehrlich dokumentieren, dass sie ein kompromittiertes Betriebssystem nicht vollständig ausgleichen kann.

Thema	Fachliche Anforderung
Master-Passwort	Pro Tresor eigenständig, niemals im Klartext speichern, niemals loggen, keine Wiederherstellung über versteckte Hintertür.
Secrets im UI	Standardmäßig maskiert, Reveal nur bewusst und temporär, Copy-Aktionen eindeutig als solche kennzeichnen.
Zwischenablage	Sensible Werte nach definierter Zeit automatisch entfernen oder überschreiben, soweit dies technisch zuverlässig möglich ist.
Auto-Lock	Tresor nach Inaktivität oder expliziter Sperre schützen; nach Entsperrung Master-Passwort des aktiven Tresors verlangen.
Logging	Keine Passwörter, Tokens, privaten Schlüssel, PFX-Passwörter oder geheimen Zusatzfelder im Klartext in Logs, Debug-Ausgaben oder Fehlermeldungen.
Recovery	Keine Backdoor. Eine spätere Recovery-Strategie darf nur als ausdrücklich eingerichteter und dokumentierter Mechanismus existieren.
Export	Unverschlüsselte Exporte nur nach ausdrücklicher Warnung und bewusster Nutzerentscheidung.

11. Recovery ohne Backdoor

Der Wunsch nach einer späteren Recovery-Strategie wird ausdrücklich aufgenommen. Das Produkt darf jedoch keine versteckte Hersteller-, Entwickler- oder Support-Hintertür enthalten. Wer ein Master-Passwort verliert, darf nicht durch einen geheimen Mechanismus heimlich wieder an die Daten gelangen können.

Für spätere Versionen sind nur offizielle, vorab bewusst eingerichtete Recovery-Varianten zulässig. Denkbar sind ein Recovery-Kit, ein gesonderter Recovery-Tresor, geteilte Recovery-Geheimnisse, Notfallumschläge oder klar dokumentierte organisatorische Verfahren. Diese Konzepte müssen gesondert bewertet werden, weil sie Komfort und Sicherheit gegeneinander abwägen.

12. Interoperabilität, Migration, Import und Export

Password-Safe-Interop bleibt ein wichtiges Ziel, wird aber als additive Schicht verstanden. Die Anwendung soll vorhandene Bestände übernehmen können, darf aber nicht versuchen, alle Grenzen und Eigenheiten von Password Safe nachzubauen. Maßgeblich ist das interne SASD-Datenmodell.

Bereich	Anforderung / Entscheidung
Password-Safe-Import	Kontrollierter Import aus .psafe3 mit Mapping, Warnungen und Importbericht.

Bereich	Anforderung / Entscheidung
Password-Safe-Export	Später nur für sauber abbildbare Inhalte; Verlusthinweise und Kompatibilitätsbericht sind Pflicht.
CSV-Export	Nur nach klarer Warnung, weil unverschlüsselte Exporte ein sehr hohes Risiko darstellen.
Browser-Import	Fachlich interessant, aber wegen Datenschutz, Browserabhängigkeit und Security erst später bewerten.
Cross-Vault-Kopie	Später sinnvoll, um Einträge aus einem anderen geöffneten Tresor zu übernehmen; nicht mit Team-Sharing verwechseln.
Zertifikatsdateien	Späterer Import von PEM/PFX/CRT/KEY als verschlüsselte Anhänge oder sichere Feldgruppen; V1 fokussiert auf Struktur und Referenzen.

13. Release- und Versionslogik

Die bisherigen Dokumente verwenden V1, V1.x und V2 nicht immer identisch. Diese überarbeitete Fassung ordnet die Anforderungen so, dass die erste echte Produktversion alltagstauglich und sicher genug wirkt, ohne zu früh Cloud-, Team- oder Enterprise-Komplexität einzubauen.

Version	Zielcharakter	Typischer Umfang
V0.x / Prototyp	Aktiver Entwicklungsstand	Grundarchitektur, WinForms-Shell, .svault, Gruppen, Tags, Suche, CRUD, Drag-and-Drop, Tests; noch kein produktives Sicherheitsprodukt.
V1.0	Runde lokale Erstversion	Mehrere Tresore, strukturierte Secret-Typen, starke Grundbedienung, sichere Anzeige, Clipboard-Autoclear, Auto-Lock, Passwortgenerator, Backups, kontrollierter .psafe3-Import, Zertifikatseintrag ohne volle PKI-Automation.
V1.x	Härtung und Produktpolitur	Master-Passwort-Änderung, erweiterte Backup-/Restore-Sicherheit, Templates, Generatorprofile, Tag-Verwaltung, Produkticon, Zertifikatsablauf-Warnungen, verbesserte Import-/Exportberichte.
V2	Deutlicher Funktionsausbau	Mehrsprachigkeit, Theme-Umschaltung, Cross-Vault-Funktionen, Passwort-Historie, TOTP, verschlüsselte Anhänge, Zertifikatsdateien, optionale Online-Prüfungen nach Opt-in.
V3/später	Optionale oder heikle Erweiterungen	Browser-Import, Passkey-/Authenticator-nahe Themen, ACME-/PKI-Automation, Team-/Cloud-/Mobile-Strategien, tiefere Enterprise-Funktionen.

14. Funktionaler Anforderungskatalog

ID	Anforderung	Version	Priorität	Erläuterung / Abgrenzung
LH-F-001	Lokale Windows-Desktop-Anwendung, offline-first und ohne Cloud-Zwang.	V1	Muss	Die Kernfunktion darf nicht von externen Diensten oder Konten abhängen.
LH-F-002	Mehrere unabhängige Tresore anlegen, öffnen, speichern, schließen und wieder öffnen.	V1	Muss	Private, SASD-interne, kundenbezogene und Archivdaten müssen trennbar bleiben.
LH-F-003	Ein Tresor ist eigene Schutzdomäne mit eigenem Master-Passwort und eigener Datei.	V1	Muss	Inhalte unterschiedlicher Tresore dürfen nicht vermischt oder gemeinsam entsperrt werden.
LH-F-004	MRU-Liste zuletzt verwendeter Tresore ohne Speicherung von Secrets oder Master-Passwörtern.	V1	Soll	Komfortfunktion ohne Verletzung der lokalen Sicherheitsgrenze.
LH-F-005	Robustes Speichern mit Schutz vor Halbspeicherung und Datenverlust.	V1	Muss	Speichervorgänge müssen konservativ und nachvollziehbar gestaltet sein.
LH-F-006	Verschlüsselte Backups und erkennbare Restore-Strategie.	V1/V1.x	Muss	Backups dürfen keine unverschlüsselten Nutzdaten erzeugen.
LH-F-007	Einträge anlegen, bearbeiten, archivieren, löschen und verschieben.	V1	Muss	Grundfunktion jeder Secret-Verwaltung.
LH-F-008	Eintragstypen Login, Database, Mail, Hosting, FTP/SFTP, Server, API, License, SecureNote, Custom.	V1	Muss	Technische Zugangsdaten brauchen fachliche Typisierung.
LH-F-009	Eigenen Eintragstyp HTTPS/TLS-Zertifikat mit strukturierten Zertifikatsfeldern bereitstellen.	V1	Muss	Zertifikate dürfen nicht nur als Freitextnotiz oder beiläufiger Zertifikatsbezug erscheinen.
LH-F-010	Privaten Schlüssel, PFX/P12-Passwort oder Key-Passphrase als geheime Felder behandeln.	V1	Muss	Zertifikatsmaterial kann genauso kritisch wie ein Passwort sein.
LH-F-011	Zertifikatsdateien als verschlüsselte Anhänge oder sichere Importobjekte verwalten.	V2	Soll	Binäranhänge erhöhen Komplexität und werden nicht in die frühe Grundversion gedrückt.

ID	Anforderung	Version	Priorität	Erläuterung / Abgrenzung
LH-F-012	Ablaufdatum und Erneuerungsfrist von Zertifikaten anzeigen und später warnen.	V1.x/V2	Soll	Nützlich für Betrieb, aber Warn-/Reminder-Logik kann nach dem Grundmodell folgen.
LH-F-013	ACME- oder automatische Zertifikatserneuerung nicht als V1-Ziel behandeln.	V3	Kann	Wäre ein eigenes Betriebs- und Integrationsprojekt.
LH-F-014	Strukturierte Zusatzfelder mit Feldtyp, Geheimkennzeichnung und Sortierung.	V1	Muss	Host, Port, Endpoints, Zertifikatsdaten und technische Metadaten dürfen nicht im Freitext verschwinden.
LH-F-015	Gruppen, Untergruppen und sichtbarer Root-/Tresorknoten.	V1	Muss	Hauptgruppen müssen klar angelegt werden können.
LH-F-016	Tags als freie Querzuordnung mit Normalisierung und Suche.	V1	Muss	Provider, Projekt, Kunde, Umgebung und Technologie sollen quer auffindbar bleiben.
LH-F-017	Suche über Titel, Benutzerkennung, Typ, Gruppen, Tags, Notizen und nicht geheime Zusatzfelder.	V1	Muss	Wiederauffindbarkeit ist Kernnutzen.
LH-F-018	Filter nach Typ und Tags, später erweiterte Tag-Verwaltung.	V1/V1.x	Soll	Alltagstauglichkeit bei vielen Einträgen.
LH-F-019	Kontextmenüs und Drag-and-Drop für Einträge und Gruppen mit Schutzabfragen.	V1	Muss	Natürliche Organisation ohne überladene Button-Oberfläche.
LH-F-020	Erstellungs- und Änderungszeitpunkte pro Eintrag.	V1	Muss	Nachvollziehbarkeit und spätere Historienfunktionen.
LH-F-021	Sensible Werte standardmäßig maskieren und nur bewusst anzeigen.	V1	Muss	Schutz gegen zufällige Offenlegung.
LH-F-022	Copy-Aktionen für sensible Werte mit Feedback und Clipboard-Autoclear.	V1	Muss	Zwischenablage ist ein praktisches Risiko.
LH-F-023	Auto-Lock / manuelles Sperren des aktiven Tresors.	V1	Muss	Arbeitsunterbrechungen dürfen keine offenen Secrets zurücklassen.
LH-F-024	Passwortgenerator mit sinnvollen Optionen und später Generatorprofilen.	V1/V1.x	Muss	Default-Passwörter werden nicht unterstützt; starke Generierung ist die bessere Alternative.

ID	Anforderung	Version	Priorität	Erläuterung / Abgrenzung
LH-F-025	Master-Passwort-Änderung mit Reverschlüsselung.	V1.x	Muss	Regulärer Passwortwechsel muss möglich werden.
LH-F-026	Warnung bei schwachen Master-Passwörtern.	V1	Muss	Sicherheitsführung ohne Zwang zur Bevormundung.
LH-F-027	Keine Secrets, Tokens, privaten Schlüssel oder Passwörter in Logs.	V1	Muss	Debug-Logging darf die Sicherheitsziele nicht unterlaufen.
LH-F-028	Keine versteckte Recovery-Backdoor.	V1	Muss	Recovery darf nur explizit, dokumentiert und vorab eingerichtet existieren.
LH-F-029	Spätere Recovery-Strategie ohne Backdoor konzipieren.	V2	Soll	Vergessene Master-Passwörter bleiben ein wichtiges Risiko, aber keine heimliche Hintertür.
LH-F-030	Kontrollierter Password-Safe-Import aus .psafe3 mit Bericht.	V1	Muss	Bestehende PWSafe-Daten sollen realistisch migrierbar sein.
LH-F-031	Password-Safe-Export nur mit sauberem Mapping und Verlusthinweisen.	V1.x/V2	Soll	Export ist schwieriger als Import und darf keine Scheinkompatibilität versprechen.
LH-F-032	CSV-Export nur nach deutlicher Warnung.	V2	Kann	Unverschlüsselte Exporte sind hochkritisch.
LH-F-033	Browser-Import gecachter Logins bewusst vertagen.	V3	Kann	Technisch und datenschutzseitig heikel.
LH-F-034	Mehrsprachigkeit mindestens Deutsch/Englisch vorbereiten.	V2	Soll	Professionalisierung, nicht V1-kritisch.
LH-F-035	Hell-/Dunkel-Umschaltung und eigenes Anwendungssymbol.	V1.x/V2	Soll	Produktreife und bessere Außendarstellung.
LH-F-036	Passwort-Historie nur kontrolliert und begrenzt einführen.	V2	Soll	Fachlich nützlich, aber sicherheitlich sensibel.
LH-F-037	TOTP-Secret-Verwaltung später prüfen.	V2	Kann	Komfortabel, aber reduziert ggf. Trennung der Faktoren.
LH-F-038	Team-Sharing, Cloud-Synchronisation und Mobile-Clients nicht in V1 aufnehmen.	Später	Nicht-Ziel	Würde Produktumfang und Sicherheitsmodell stark verändern.

15. Nichtfunktionale Anforderungen

Qualitätsziel	Anforderung
Sicherheit	Sichere Standardbedienung hat Vorrang vor Komfortabkürzungen. Risikoarme Defaults sind Pflicht.
Nachvollziehbarkeit	Datenmodell, Bedienung und Versionslogik müssen in Dokumentation und UI verständlich bleiben.
Wartbarkeit	Code und Dokumentation sollen denselben Begriffshaushalt verwenden; fachliche Logik darf nicht unkontrolliert in UI-Klassen wachsen.
Erweiterbarkeit	Zertifikatsanhänge, TOTP, Cross-Vault-Funktionen und weitere Interop sollen später ohne Bruch anschließbar sein.
Performance	Normale lokale Nutzung muss flüssig bleiben. Extreme Optimierung ist weniger wichtig als Sicherheit und Klarheit.
Testbarkeit	Sicherheits-, Speicher-, Such-, Import- und Organisationslogik müssen durch Tests absicherbar sein.
Dokumentationsqualität	Lastenheft, Pflichtenheft, Architektur, Roadmap und README müssen konsistent bleiben.

16. Bewusste Nicht-Ziele und vertagte Themen

Die folgenden Punkte sind nicht vergessen, sondern bewusst abgegrenzt. Sie dürfen in späteren Versionen erneut bewertet werden, gehören aber nicht in den frühen Kern oder nicht ohne eigenes Konzept in die Umsetzung.

Thema	Entscheidung	Begründung
Vollständiger 1:1-Klon von Password Safe	Nicht Ziel	Das interne Modell muss stärker sein als das Fremdformat.
Gruppenweite Default-Passwörter	Nicht umsetzen	Fördert Passwortwiederverwendung; ersetzt durch Generatorprofile und Vorlagen.
Virtuelle Tastatur	Nicht priorisieren	Begrenzter Sicherheitsgewinn bei hohem Bedienaufwand.
Cloud-Synchronisation	Vertagt	Ändert Sicherheitsmodell und Betrieb deutlich.
Team-Sharing / Rollenrechte	Vertagt	Eigenes Produkt- und Sicherheitskonzept erforderlich.
Mobile Clients	Vertagt	Eigene Plattform- und Sync-Fragen.
Browser-Autofill	Vertagt	Hohe Sicherheits- und Integrationskomplexität.
Automatisierte ACME-/PKI-Verwaltung	Später prüfen	Sinnvoll, aber deutlich mehr als Zertifikatsdokumentation.
Rich-Text/RTF in Notizen	Vertagt	Erhöht Komplexität, Angriffsfläche und Interop-Probleme.

17. Aktueller Prototypstand

Der vorhandene Repository-Stand bildet bereits einen erheblichen Teil des Fundaments ab. Vorhanden sind eine .NET-8-WinForms-Anwendung, getrennte Schichten, internes .svault-Format, verschlüsseltes Speichern und Laden, Gruppen, Untergruppen, Tags, Suche, Listen- und Detailansicht, Eintrags- und Gruppenbearbeitung, Drag-and-Drop, Passwortstärke-Bewertung, Debug-Logging und Tests.

Gleichzeitig bleibt das Repository ein aktiver Entwicklungsstand. Vor einer produktiven Nutzung mit echten kritischen Geheimnissen müssen insbesondere Clipboard-Schutz, Auto-Lock, Passwortgenerator, Password-Safe-Import, Backup-/Restore-Härtung, Recovery-Entscheidungen und Zertifikatsverwaltung sauber abgeschlossen und getestet werden.

Bereich	Stand laut bisheriger Einordnung
Architektur / Solution	Grundstruktur Domain, Application, Security, Storage, Interop, WinForms und Tests vorhanden.
Tresor-Lifecycle	Anlegen, Öffnen, Speichern und verschlüsseltes .svault-Format vorhanden beziehungsweise angelegt.
Organisation	Gruppen, Tags, Root-Logik, Drag-and-Drop und Schutzdialoge in wesentlichen Teilen vorhanden.
Alltagssicherheit	Clipboard-Autoclear, Auto-Lock, Generator und Interop noch Härtungs-/Abschlussbereich.
Zertifikatsverwaltung	Als Anforderung jetzt ausdrücklich aufgenommen; konkrete UI-/Datenmodell-Erweiterung folgt im Pflichtenheft.

18. Abnahmekriterien für V1

V1 ist fachlich abnahmefähig, wenn der lokale Kern sicher und alltagstauglich funktioniert. Die Abnahme erfolgt nicht über einzelne Menüpunkte, sondern über überprüfbare Nutzungsszenarien.

1. Ein neuer .svault-Tresor kann angelegt, gespeichert, geschlossen, wieder geöffnet und weiterbearbeitet werden, ohne Datenverlust oder Vermischung mit anderen Tresoren.
2. Mehrere unabhängige Tresore können nacheinander genutzt werden; Master-Passwörter und Inhalte bleiben getrennt.
3. Einträge für Login, Datenbank, Mail, FTP/SFTP, Server, API, Lizenz, SecureNote, Custom und HTTPS/TLS-Zertifikat können sinnvoll erfasst und wiedergefunden werden.
4. Zertifikatseinträge besitzen strukturierte Felder für Domain/CN/SAN, Aussteller, Fingerprint, Laufzeit, Ablageort, Erneuerung und geheime Bestandteile wie Key-Passphrase oder PFX-Passwort.
5. Gruppen, Untergruppen, Tags, Suche, Filter, Kontextmenüs und Drag-and-Drop funktionieren nachvollziehbar und schützen vor riskanter Fehlbedienung.
6. Geheime Werte sind standardmäßig verborgen, lassen sich bewusst anzeigen oder kopieren und verbleiben nicht dauerhaft in der Zwischenablage.
7. Der Tresor kann manuell und automatisch gesperrt werden; sichtbare Secrets werden beim Sperren entfernt oder maskiert.
8. Backups bleiben verschlüsselt; Speicher- und Fehlerfälle werden verständlich gemeldet.
9. Password-Safe-Import erzeugt einen nachvollziehbaren Importbericht und verschluckt nicht stillschweigend nicht abbildbare Informationen.
10. Logs, Fehlermeldungen und Debug-Ausgaben enthalten keine Klartext-Secrets, Tokens, privaten Schlüssel oder Master-Passwörter.

19. Offene Punkte für das anschließende Pflichtenheft

Das Lastenheft legt die fachliche Richtung fest. Für das Pflichtenheft beziehungsweise die nächste technische Überarbeitung sind vor allem folgende Punkte zu präzisieren:

- Exaktes Datenmodell für HTTPS/TLS-Zertifikatseinträge, inklusive Feldnamen, Sensitivitätskennzeichen und Beziehung zu Server-/Domain-/Hosting-Einträgen.
- Entscheidung, ab wann verschlüsselte Anhänge beziehungsweise Zertifikatsdateien im Tresor gespeichert werden dürfen.
- Konkretes UI-Konzept für Zertifikatsdetails, Ablaufwarnungen, Deployment-Bezüge und geheime Schlüsselbestandteile.
- Detailkonzept für Clipboard-Autoclear, Auto-Lock, Master-Passwort-Änderung und Passwortgenerator.
- Mapping-Regeln für Password-Safe-Import und spätere Exportberichte.
- Recovery-Konzept ohne Backdoor mit klarer Abwägung von Sicherheit, Bedienbarkeit und organisatorischem Notfallbedarf.
- Abgleich zwischen Repository-README, Roadmap, Lastenheft, Pflichtenheft und Architektur, damit keine widersprüchlichen V1/V1.x-Grenzen bleiben.

20. Quellen- und Entscheidungsregister

Dieses Register nennt die wichtigsten einbezogenen Unterlagen und Entscheidungen. Es ersetzt keine externe Sicherheitsprüfung und keine formale Compliance-Freigabe.

Quelle / Bereich	Einfluss auf dieses Lastenheft
Fortgeschriebenes Lastenheft	Grundstruktur, Zielbild, vollständiges Ideen- und Versionsregister.
Pflichtenheft und Architekturkonzept	Abgrenzung zwischen fachlichem Bedarf und technischer Umsetzung; Schichten- und Versionslogik.
Architekturdokument V1	Ist-/Soll-Abgleich des Repository-Stands und offene V1-Bausteine.
Feature-Collation Passwortmanager	Abgrenzung zu KeePassXC, Bitwarden, 1Password, Keeper und anderen Produkten; Migrations-, Sicherheits- und Betriebsfragen.
Roadmap	Milestones, bereits erreichte Funktionen und noch offene V1-Schritte.
Projektchats	Spätere Entscheidungen, insbesondere Recovery ohne Backdoor, Zertifikatsverwaltung und klare Trennung von Strategie und Entwicklungsarbeit.
GitHub-Repository	Aktueller README-Stand, Projektstruktur, Statushinweise, Build-/Test-Kommandos und Sicherheitswarnung.

21. Schlussbemerkung

Das überarbeitete Lastenheft hält die gereifte Produktidee fest: ein lokaler, sicherheitsbewusster Desktop Secret Manager für mehrere verschlüsselte Tresore, der technische Zugänge, klassische Logins und nun ausdrücklich auch HTTPS/TLS-Zertifikatsinformationen strukturiert verwalten kann.

Die wichtigste fachliche Linie bleibt: V1 soll klein genug bleiben, um stabil, testbar und glaubwürdig zu werden, aber vollständig genug, um im Alltag als ernsthafter lokaler Secret Manager nutzbar zu sein.

Erweiterungen wie Cloud, Team-Sharing, Browser-Autofill, mobile Clients oder PKI-Automation werden nicht vergessen, aber bewusst erst nach einem stabilen lokalen Fundament bewertet.